



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) «Компьютерные науки и прикладная математика»

Образовательный центр Института № 8

Группа М8О-407Б-22 Направление подготовки 01.03.02 «Прикладная математика и информатика»

Профиль «Информатика»

Квалификация бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: Микросервисное веб-приложение для управления внутренними задачами, проектами и коммуникацией команды

Автор ВКРБ Кострюков Евгений Сергеевич (Кост)

Руководитель Кузнецова Светлана Валентиновна (Кузнецова)

Консультант ()

Рецензент ()

К защите допустить

И.о. Директора Образовательного
Центра Института №8

Крылов Сергей Сергеевич (Крылов)

12 мая 2026 г.

РЕФЕРАТ

Выпускная квалификационная работа бакалавра содержит около 50 страниц основного текста, 4 рисунка, 5 листингов фрагментов программного кода и конфигураций в главе о результатах работы, 12 наименований использованных источников.

МИКРОСЕРВИСНАЯ АРХИТЕКТУРА, ВЕБ-ПРИЛОЖЕНИЕ, УПРАВЛЕНИЕ ЗАДАЧАМИ, REST API, JWT, SPRING BOOT, REACT, POSTGRESQL, DOCKER

Объект разработки — программная система учёта проектов и поручений с веб-интерфейсом, построенная как набор взаимодействующих сервисов с отдельными базами данных.

Цель работы — получить работоспособный прототип (MVP) системы, в которой сотрудники могут совместно вести проекты и задачи, обмениваться комментариями и файлами, получать уведомления о значимых событиях и строить отчёты по периоду.

Для достижения цели решены следующие задачи:

1. Сформулированы функциональные и нефункциональные требования к программному комплексу и выбрана микросервисная декомпозиция на сервис аутентификации, сервис задач и проектов, сервис аналитики и уведомлений [1].
2. Спроектированы модели данных и соглашения обмена по REST между клиентом и серверами, а также между сервисами для внутренних уведомлений [2].
3. Реализованы backend-компоненты на Java 17 с использованием Spring Boot 3 [3], включая проверку прав доступа и хранение вложений во внешнем объектном хранилище MinIO [4].
4. Реализован одностраничный веб-клиент на React 18 и Vite с маршрутизацией экранов, формами ввода и отображением аналитики [5].

5. Подготовлены сценарии развёртывания через Docker Compose [6], журналирование запросов и автоматизированное тестирование с прогоном в среде непрерывной интеграции.

Основное содержание работы — в обосновании выделения границ сервисов, описании реализованных API, моделей безопасности на основе JWT [7] и ролей, а также в главе о результатах работы, где приводятся фрагменты Docker Compose, исходного кода backend и файла конвейера CI.

Результат пригоден для использования в учебно-исследовательской практике как основа корпоративной системы планирования работ и может быть доработан под конкретные регламенты заказчика.

Практическая ценность связана с возможностью локального развёртывания без привязки к стороннему облачному SaaS, воспроизводимостью окружения и явным разделением зон ответственности между компонентами, что облегчает сопровождение и поэтапное расширение функциональности.

СОДЕРЖАНИЕ

РЕФЕРАТ	2
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	7
ВВЕДЕНИЕ	8
1 ОБЩАЯ ЧАСТЬ.....	11
1.1 Актуальность автоматизации учёта задач и проектов в организации	11
1.2 Сравнение подходов: монолит, модульный монолит и микросервисы	12
1.3 Обзор типовых средств класса task / project management	13
1.4 Требования к разрабатываемой системе и критерии приёмки прототипа	14
1.5 Вопросы информационной безопасности и разграничения доступа	15
1.6 Постановка задачи и формулировка технического задания.....	16
2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ЗАДАЧИ.....	18
2.1 Общая структура репозитория и состав исполняемых модулей..	18
2.2 Проектирование границ сервисов и контрактов REST API	20
2.3 Модели баз данных и согласованность при распределённом хранении.....	22
2.4 Сервис аутентификации и управления учётными записями	27
2.5 Сервис проектов, задач, комментариев, вложений и истории	29
2.6 Сервис уведомлений, отчётности и экспорта данных.....	31
2.7 Веб-клиент: маршруты, состояние, взаимодействие с API	33

2.8 Контейнеризация, конфигурирование окружения и журналирование	35
2.9 Тестирование и конвейер непрерывной интеграции.....	36
3 РЕЗУЛЬТАТ РАБОТЫ.....	38
3.1 Описание стенда и фрагмент Docker Compose	38
3.2 Контроллер проектов и делегирование прикладным сервисам ...	40
3.3 Ограничение выборки задач для роли исполнителя	41
3.4 Внутренний вызов создания уведомления в сервисе аналитики	42
3.5 Файл конвейера непрерывной интеграции.....	43
3.6 Выводы.....	45
ЗАКЛЮЧЕНИЕ.....	46
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	47
ПРИЛОЖЕНИЕ.....	49

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Микросервис — автономно развёртываемый компонент программной системы с собственным жизненным циклом и, как правило, отдельным хранилищем данных, взаимодействующий с другими компонентами по сети

REST API — интерфейс взаимодействия программ по протоколу HTTP, опирающийся на ресурсы, идентифицируемые URI, и представление состояния в формате JSON или другом согласованном формате

JWT (JSON Web Token) — компактное представление набора утверждений, передаваемое между сторонами для подтверждения подлинности субъекта при условии проверки цифровой подписи или имитовставки

RBAC (ролевая модель доступа) — способ назначения полномочий через фиксированные роли пользователей с последующей проверкой разрешённых действий в коде приложения

MVP (минимально жизнеспособный продукт) — версия системы с ограниченным, но достаточным для демонстрации ценности набором функций, пригодная для опытной эксплуатации и сбора обратной связи

Объектное хранилище (в работе — MinIO, совместимое с S3) — сервис для хранения двоичных объектов (файлов) с доступом по HTTP API, используемый для вынесения тяжёлых данных за пределы реляционной базы задач

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

API — программный интерфейс приложения (Application Programming Interface)

HTTP — протокол передачи гипертекста

JSON — текстовый формат обмена данными (JavaScript Object Notation)

JWT — токен в формате JSON Web Token

CRUD — набор операций создания, чтения, обновления и удаления (create, read, update, delete)

SSE — отправка событий по HTTP от сервера к клиенту (Server-Sent Events)

CORS — совместное использование ресурсов разных источников (Cross-Origin Resource Sharing)

БД — база данных

СУБД — система управления базами данных

ORM — объектно-реляционное отображение (Object-Relational Mapping)

CI — непрерывная интеграция (Continuous Integration)

ВВЕДЕНИЕ

Планирование работ, фиксация договорённостей и прозрачность статусов поручений давно вышли за рамки отдельных таблиц и переписки в мессенджере: при росте числа участников растёт и цена ошибки в учёте. Компании нередко опираются на облачные сервисы класса управления задачами, однако для ряда организаций важны контроль над данными, предсказуемая стоимость владения и возможность встроить собственные регламенты согласования. Отсюда следует интерес к решениям, которые можно развернуть на собственной инфраструктуре и развивать поэтапно.

Параллельно закрепились практика дробления серверной части на независимые сервисы: это не универсальный ответ на все задачи, но при ясных границах предметных областей такой подход упрощает масштабирование отдельных узких мест и локализует изменения при доработках. Для учебного проекта выпускной квалификационной работы выбрана именно микросервисная композиция: сервис идентификации пользователей, сервис проектов и задач, сервис уведомлений и отчётности, при этом у каждого backend-сервиса — своя база PostgreSQL [8], а файлы вложений выносятся в объектное хранилище.

Цель работы — разработать и описать прототип веб-приложения для совместной работы с проектами и задачами, включающий регистрацию и вход, роли исполнителя, менеджера и администратора, операции над проектами и участниками, полный цикл задачи с комментариями и вложениями, журнал изменений, уведомления и аналитические отчёты с выгрузкой. Для достижения цели были поставлены и решены следующие задачи:

1. Проанализированы существующие классы систем и обоснован выбор микросервисной архитектуры для прототипа.

2. Сформулированы требования к функциям безопасности: JWT, хранение паролей в виде хеша BCrypt, проверки прав на уровне сервисов, защита внутренних вызовов между сервисами.
3. Спроектированы модели данных трёх баз и связь «участник проекта — допустимый исполнитель задачи».
4. Реализованы три backend-приложения на Spring Boot 3 (Java 17) с разделением HTTP-слоя и прикладной логики.
5. Реализован общий модуль проверки JWT для серверов ресурсов задач и аналитики.
6. Реализован веб-клиент на React 18 с маршрутизацией основных разделов и обработкой ошибок API.
7. Настроены Docker Compose-сценарии, журналирование и воспроизводимый запуск.
8. Подготовлены автоматические тесты backend и клиента, а также сквозные проверки, включённые в конвейер CI.

При реализации применялись следующие средства разработки и сопровождения:

1. Java 17 и Spring Boot 3 — серверная реализация REST API, безопасность Spring Security, доступ к данным Spring Data JPA.
2. PostgreSQL 16 — реляционное хранение по принципу «отдельная БД на сервис».
3. MinIO — хранение двоичных вложений задач.
4. React 18, Vite, React Router — клиентское одностраничное приложение.
5. Vitest и Testing Library — модульные проверки клиентских функций и хуков.

6. Docker и Docker Compose — упаковка сервисов и зависимостей, подъём стенда одной командой.

7. Playwright и GitHub Actions — сквозные сценарии и автоматический прогон при изменениях в репозитории.

По итогам работы получен развёртываемый программный комплекс: после регистрации и входа пользователь ведёт проекты и задачи, обменивается комментариями и файлами, получает уведомления о событиях, просматривает сводные показатели за выбранный период и при необходимости выгружает отчёт в файл. В общей части рассматриваются предпосылки и постановка задачи; в разделе проектирования и реализации приводятся архитектурные решения, описание сервисов, клиента и инфраструктуры; в разделе о результатах работы — фрагменты конфигурации и кода, подтверждающие принятые инженерные решения, и выводы; заключение обобщает итоги и возможные направления доработки.

1 ОБЩАЯ ЧАСТЬ

1.1 Актуальность автоматизации учёта задач и проектов в организации

Работа над проектом редко укладывается в один канал общения: договорённости фиксируются в переписке, сроки — в календаре, артефакты — в общей папке, а статус «кто что делает сейчас» приходится собирать вручную на совещании. Чем шире круг участников и чем больше параллельных направлений, тем быстрее расходятся версии правды о состоянии работ. Ошибка в учёте исполнителя или пропущенное изменение статуса задачи оборачиваются повторной работой, срывом промежуточной вехи и спором о том, кто и когда принял решение.

Отдельная боль — преемственность: новый сотрудник не видит истории обсуждений по задаче, если она жила в личных чатах; руководитель не может быстро получить срез по загрузке команды без выгрузок и сводных таблиц. Поэтому организации всё чаще переходят к специализированным системам учёта поручений, где карточка задачи становится единым местом хранения описания, обсуждения, вложений и журнала изменений.

Вместе с тем не каждому подходит готовый облачный сервис с фиксированной моделью ролей и хранением данных у третьей стороны. Внутренние регламенты, требования к размещению информации и желание не зависеть от внешнего тарифа подталкивают к разработке собственного приложения, разворачиваемого на контролируемой площадке. Для учебной выпускной работы такой выбор дополнительно позволяет пройти полный цикл: от постановки требований до контейнерного развёртывания и автоматических проверок.

Итоговая постановка сводится к необходимости прототипа, в котором команда ведёт проекты и задачи в одном интерфейсе, видит уведомления о

значимых событиях и может получить отчёт за период без ручной агрегации в табличном редакторе.

1.2 Сравнение подходов: монолит, модульный монолит и микросервисы

Серверную часть типичного веб-приложения можно собрать как единый исполняемый модуль (монолит): один процесс, одна кодовая база, одна схема БД, общие транзакции. Плюсы знакомы — простой старт, отладка в одном месте, отсутствие сетевых сбоев между «внутренними» компонентами. Минусы проявляются позже: рост команды и функционала удлиняет сборку и регрессионное тестирование, любое изменение тянет риск побочных эффектов в смежных подсистемах, масштабировать приходится приложение целиком, даже если нагрузка пришла только на один участок.

Промежуточный вариант — модульный монолит: границы предметных областей соблюдаются в коде и на уровне пакетов, но развёртывание остаётся единым. Это снижает связанность, но не даёт независимого выпуска и отдельного масштабирования модулей без перехода к нескольким процессам.

Микросервисная декомпозиция выносит части системы в отдельные сервисы с собственным жизненным циклом и, в идеале, собственной базой данных. Цена — согласование контрактов API, распределённые операции, наблюдаемость и отказоустойчивость на уровне сети. Выигрыш — можно обновлять и масштабировать узкий участок (например, выдачу отчётов), не трогая остальное; команда может разделить ответственность по сервисным границам.

Для прототипа системы управления задачами естественно выделить хотя бы три области: учёт пользователей и аутентификация; проекты, задачи, комментарии и файлы; уведомления и агрегированная аналитика. Они по-разному меняются по времени и по нагрузке: поток уведомлений и отчётов может пульсировать, тогда как ядро задач остаётся центральным узлом.

Разнесение по сервисам упрощает дальнейшее развитие и демонстрирует типичный промышленный приём, хотя для совсем малой команды монолит оставался бы экономически оправданным компромиссом.

1.3 Обзор типовых средств класса task / project management

Рынок предлагает десятки решений с пересекающимся набором функций; ниже — обзорный ориентир без претензии на исчерпание. Зарубежные облачные продукты вроде Jira, Asana или Monday.com глубоко закрывают гибкие процессы, шаблоны workflow, интеграции с системами контроля версий и CI; они ориентированы на крупные команды и часто опираются на платную подписку и хранение данных у поставщика. Trello и аналоги с доской карточек выигрывают наглядностью и низким порогом входа, но слабее в части жёстких регламентов и встроенной отчётности для произвольных полей.

В российском сегменте встречаются как облачные сервисы, так и коробочные варианты совместной работы, иногда в составе порталов или CRM: например, «Битрикс24» совмещает задачи, чаты и документооборот; для части организаций это удобно, для части — избыточно и тяжеловесно. YouTrack и аналоги ближе к разработческим командам, акцент на связке с репозиторием и трекинг времени. GitHub/GitLab Issues минималистичны относительно полноформатных PM-систем, зато органично вписаны в процесс разработки.

Общие закономерности: почти везде есть проект (или эквивалент), задача, исполнитель, статус, комментарии; вложения и история изменений встречаются часто, но реализованы по-разному; уведомления и отчёты зависят от тарифа и экосистемы. Разрабатываемый в работе прототип сознательно не копирует интерфейс конкретного вендора: цель — локально разворачиваемый учебно-прикладной комплекс с явным разделением сервисов и открытым для доработки кодом, а не конкуренция с коммерческим продуктом по числу интеграций.

1.4 Требования к разрабатываемой системе и критерии приёмки прототипа

Функциональные требования сформулированы относительно роли пользователя и жизненного цикла данных. Пользователь регистрируется и входит в систему; первому зарегистрированному назначается административная роль для дальнейшего управления ролями остальных. Администратор может менять роли; менеджер и исполнитель получают разные объёмы прав при работе с проектами и задачами. Для каждого проекта ведётся список участников; задачу можно отфильтровать по проекту, статусу и исполнителю. Исполнитель назначается только из числа участников проекта. К задаче добавляются комментарии, файлы загружаются в хранилище и скачиваются через API; фиксируется история изменений. При значимых событиях формируются уведомления; пользователь просматривает ленту и может отмечать прочтение. Раздел аналитики выдаёт сводку за период и позволяет выгрузить отчёт в файл табличного формата.

Нефункциональные требования для MVP включают: развёртывание через Docker Compose с отдельными контейнерами СУБД для каждого backend-сервиса; использование JWT для защищённых запросов; время отклика в пределах, приемлемых для интерактивной работы на тестовом стенде; наличие журналов запросов для отладки; воспроизводимый набор автоматических тестов (модульные, интеграционные, сквозные) и прогон в CI при изменении репозитория.

Критерии приёмки прототипа сформулированы проверяемо:

1. Успешная регистрация, вход, смена пароля и редактирование профиля; выдача и проверка JWT.
2. Создание проекта, добавление и удаление участников; создание, редактирование и удаление задач с корректной фильтрацией.

3. Ошибка или отказ API при попытке назначить исполнителем пользователя вне состава проекта; сообщение об ошибке доходит до интерфейса.
4. Загрузка, предпросмотр (где применимо), скачивание и удаление вложения; метаданные в БД, двоичные данные — в объектном хранилище.
5. Запись истории изменений по задаче; отображение ленты комментариев.
6. Появление уведомлений при событиях из сервиса задач; невозможность прочитать чужие уведомления под чужой учётной записью.
7. Построение отчёта за интервал дат и выгрузка CSV; отсутствие падения при пустых данных в периоде.
8. Подъём всего стенда по описанию из репозитория; прохождение тестов в конвейере непрерывной интеграции на эталонной конфигурации.

1.5 Вопросы информационной безопасности и разграничения доступа

Даже учебный стенд должен демонстрировать базовые приёмы защиты, иначе прототип нельзя считать приближенным к практике. Пароли не хранятся открытым текстом: применяется хеширование BCrypt. Клиент после входа получает JWT; серверы задач и аналитики принимают решения о доступе, проверяя подпись и срок действия токена и извлекая идентификатор пользователя и роли. Разграничение по ролям (администратор, менеджер, исполнитель) согласовано с бизнес-правилами: исполнитель видит только проекты, где он состоит в участниках; менеджер и администратор получают более широкий доступ в соответствии с реализованной политикой.

Доступ к файлам вложений не даётся «прямой ссылкой» из объектного хранилища минуя API: клиент запрашивает поток у авторизованного endpoint сервиса задач. Межсервисный вызов, которым сервис задач создаёт уведомление в сервисе аналитики, защищается секретным ключом, известным только этим компонентам, чтобы внешний клиент не мог

подделать внутреннее сообщение. В браузерной среде разработки настроен CORS для разрешённого origin клиента совместно с проверкой JWT — иначе одностраничное приложение на другом порту не смогло бы вызывать API.

Ограничения MVP честно учитываются: нет отдельного аудита действий администратора на уровне неизменяемого журнала, нет двухфакторной аутентификации и политик парольной сложности сверх базовой валидации, нет защиты от распределённых атак на уровне внешнего шлюза — эти темы оставлены для последующего развития и не входят в минимальный объём выпускной работы.

1.6 Постановка задачи и формулировка технического задания

Требуется разработать прототип микросервисного веб-приложения для совместной работы с внутренними проектами и задачами. Система должна состоять из сервиса аутентификации и профилей пользователей, сервиса проектов и задач (включая комментарии, вложения и историю), сервиса уведомлений и отчётности, а также веб-клиента. Каждый backend-сервис использует отдельную базу PostgreSQL; файлы вложений размещаются в объектном хранилище MinIO. Обмен между клиентом и серверами — по REST и JSON; для защиты пользовательских операций применяется JWT.

Техническое задание на проектирование и реализацию включает следующие положения:

1. Реализовать регистрацию, аутентификацию, управление профилем и смену пароля; предусмотреть выдачу списка пользователей для назначения исполнителей и административного интерфейса ролей.
2. Реализовать CRUD проектов и управление участниками проекта; CRUD задач с фильтрами; проверку принадлежности исполнителя к проекту.
3. Реализовать комментарии к задачам, загрузку и выдачу вложений через API с хранением содержимого во внешнем хранилище; вести историю изменений атрибутов задачи.

4. Реализовать приём внутренних уведомлений сервисом аналитики, выдачу ленты пользователю, отметку прочтения и поток событий для обновления интерфейса; сформировать отчёты по сводке, разрезам «по проекту» и «по исполнителю» с фильтром периода и выгрузкой CSV.
5. Развернуть компоненты в Docker Compose, обеспечить журналирование запросов и воспроизводимый сценарий запуска для разработчика.
6. Подготовить автоматические тесты backend и клиента, сквозные сценарии и включить их в конвейер CI при публикации изменений.

Выполнение перечисленных пунктов считается достаточным для признания цели выпускной квалификационной работы достигнутой на уровне работоспособного MVP, пригодного для демонстрации и дальнейшей доработки под эксплуатацию в контуре конкретной организации.

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ЗАДАЧИ

2.1 Общая структура репозитория и состав исполняемых модулей

На рисунке 1 показана логическая архитектура программной системы TaskBoard: три backend-сервиса, по одному экземпляру PostgreSQL на каждый из них, объектное хранилище MinIO и браузерный клиент, а также основные направления HTTP-взаимодействия между компонентами.

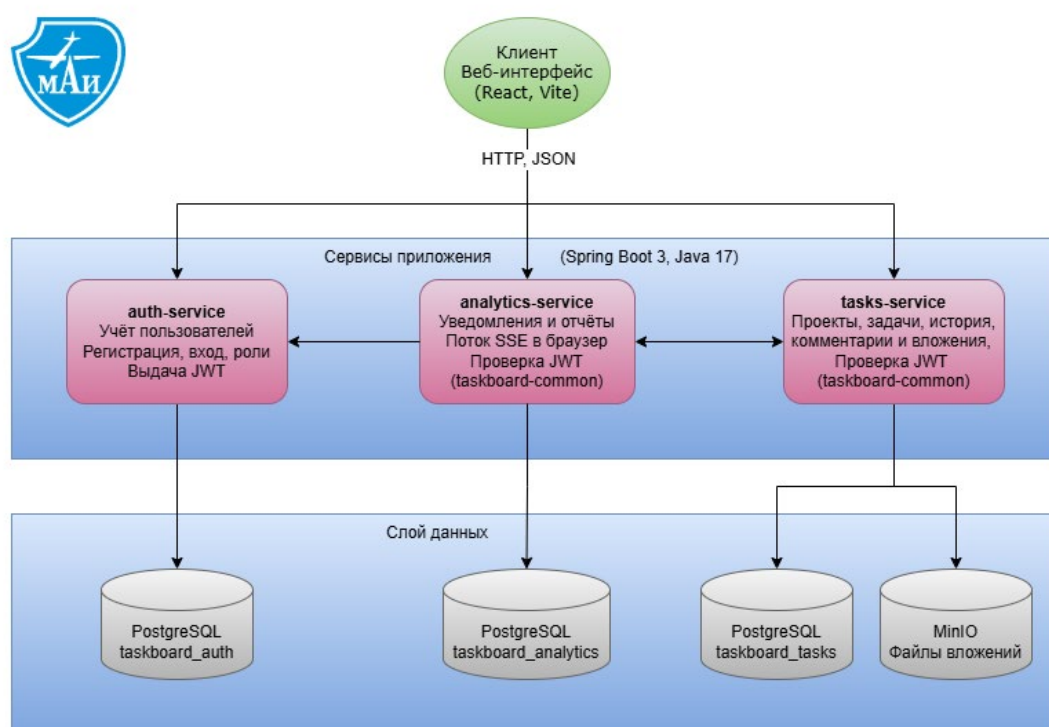


Рисунок 1 – Логическая архитектура программной системы TaskBoard

На рисунке 1 одновременно видны и граница сервиса учётных записей, и путь запросов браузера к трём API, и то, как модуль задач обращается к MinIO, не открывая прямой доступ к файлам из клиента.

Исходный код собран в одном репозитории, но исполняемые части разделены так, чтобы их можно было собирать, версионировать и разворачивать по отдельности. Корневой файл сборки Maven `pom.xml` объединяет модули `taskboard-common`, `tasks-service` и `analytics-service`: общая библиотека подключается двумя сервисами как зависимость, чтобы не копировать разбор JWT в двух серверах ресурсов. Сервис `auth-service` оформлен отдельным Maven-проектом со своим `pom.xml`, поэтому его жизненный цикл и настройки ведутся автономно, хотя контейнеры всех трёх backend-приложений поднимаются из одного описания Docker Compose и стартуют согласованно.

Модуль `taskboard-common` не является самостоятельным процессом: это подключаемая библиотека (JAR) с классом `ResourceServerJwtParser`, который по общему секрету проверяет подпись токена HS256 и извлекает из JWT числовой идентификатор пользователя и перечень ролей. Его используют `tasks-service` и `analytics-service`; выдачу токена целиком оставляют за `auth-service`.

Каталог `frontend` содержит одностраничное приложение на React 18 со сборкой Vite. В режиме разработки оно обслуживается отдельным dev-сервером на порту, не совпадающем с портами API, поэтому на backend включена политика CORS совместно с проверкой JWT — иначе браузер заблокировал бы межсайтовые запросы даже при корректном токене.

Рядом с кодом располагаются материалы для сопровождения: документация по API, ER-диаграммы баз, вспомогательные сценарии и набор типовых HTTP-запросов для ручной проверки; при локальном запуске журналы сервисов пишутся в отдельные файлы с ротацией; проверка

качества автоматизирована сценариями тестов и конвейером CI. Так отделён прикладной код от эксплуатационных артефактов.

В типовой конфигурации Docker Compose процесс auth-service слушает порт 8081, tasks-service — 8082, analytics-service — 8083; клиент разработчика на Vite — 3000. Номера портов задаются конфигурацией окружения, а не зашиваются в бизнес-логику: при переносе стенда меняют переменные, сохраняя неизменными относительные пути API.

2.2 Проектирование границ сервисов и контрактов REST API

Граница между auth-service и остальными сервисами проведена по критерию владения данными: пароли, адреса электронной почты, привязка учётной записи к ролям и признак активности хранятся только в базе taskboard_auth. Идентификатор пользователя из этой базы затем попадает в JWT и повторяется в полях создателя, исполнителя и автора комментария в сервисе задач без дублирования логина и почты в каждой строке — так уменьшается объём данных и отпадает необходимость в распределённых соединениях таблиц из разных СУБД; для согласования достаточно токена и при необходимости отдельного запроса справочника пользователей.

В tasks-service сосредоточены сущности проекта, состав участников, задачи, обсуждения, метаданные вложений и журнал правок. Сводки по всей организации он не считает: агрегаты и персональные уведомления оставлены в analytics-service, куда из модуля задач уходят только короткие сообщения о событии по заранее согласованному внутреннему формату.

Сервис analytics-service хранит уведомления, адресованные конкретному пользователю по числовому идентификатору из JWT, и при расчёте отчётности опрашивает сервис задач по read-only интерфейсу за срезом данных, нужным для статистики. Запись фактов в базе задач и построение отчётов разделены по процессам: отчётный модуль можно менять чаще, не утяжеляя транзакции в карточках поручений.

Внешний контракт для браузера выдержан в стиле REST: ресурсы именуются во множественном числе (например, пути вида `/api/projects` и `/api/tasks`), идентификаторы сущностей передаются в URI, фильтры списков — в строке запроса, тела создания и обновления — в JSON. Для защищённых операций клиент передаёт заголовок `Authorization` с носителем `Bearer` и самим JWT. Сообщения об ошибках в ответе API несут конкретику отказа; клиент выводит их как есть, без собственной расшифровки.

Вызов создания внутреннего уведомления из `tasks-service` в `analytics-service` недоступен браузеру: запрос должен сопровождаться секретом, известным только этим сервисам (в конфигурации задаётся как `NOTIFICATIONS_INTERNAL_KEY`), иначе обработчик отклоняет обращение. В теле фиксируются тип события, заголовок и текст; идентификатор получателя для внешнего клиента не подменяется произвольным значением из тела — он выводится из контекста безопасности либо задаётся доверенным кодом при межсервисном вызове.

Чтобы не опрашивать весь список уведомлений после каждого действия, подключена отдельная конечная точка по протоколу `Server-Sent Events`: HTTP-сессия держится открытой, сервер при появлении записи присылает текст события; для счётчика непрочитанного этого достаточно, `WebSocket` не понадобился.

Сводная группировка внешних маршрутов по назначению:

1. `auth-service` — регистрация, вход, проверка токена, чтение и изменение профиля, смена пароля, каталог пользователей, административное назначение ролей.
2. `tasks-service` — проекты и участники, задачи с фильтрацией списков, комментарии, загрузка и выдача вложений, журнал изменений задачи.

3. analytics-service — уведомления с фильтрами, неп прочитанные, отметка прочтения, поток SSE, сводные и разрезные отчёты, выгрузка табличного файла.

Каждая группа в типовом стенде обслуживается своим хостом и портом; набор запросов дополнительно фиксируется в документации проекта для ручной и автоматической проверки.

2.3 Модели баз данных и согласованность при распределённом хранении

Данные разнесены по трём независимым базам PostgreSQL, что соответствует границам микросервисов и не допускает прямых внешних ключей между СУБД разных приложений. Числовые ключи пользователей и проектов намеренно совпадают с теми, что фигурируют в JWT и в строках, созданных сервисом задач: это договорённость между сервисами, а не общий внешний ключ в одной СУБД.

На рисунке 2 представлена структура базы taskboard_auth, обслуживаемой сервисом аутентификации.

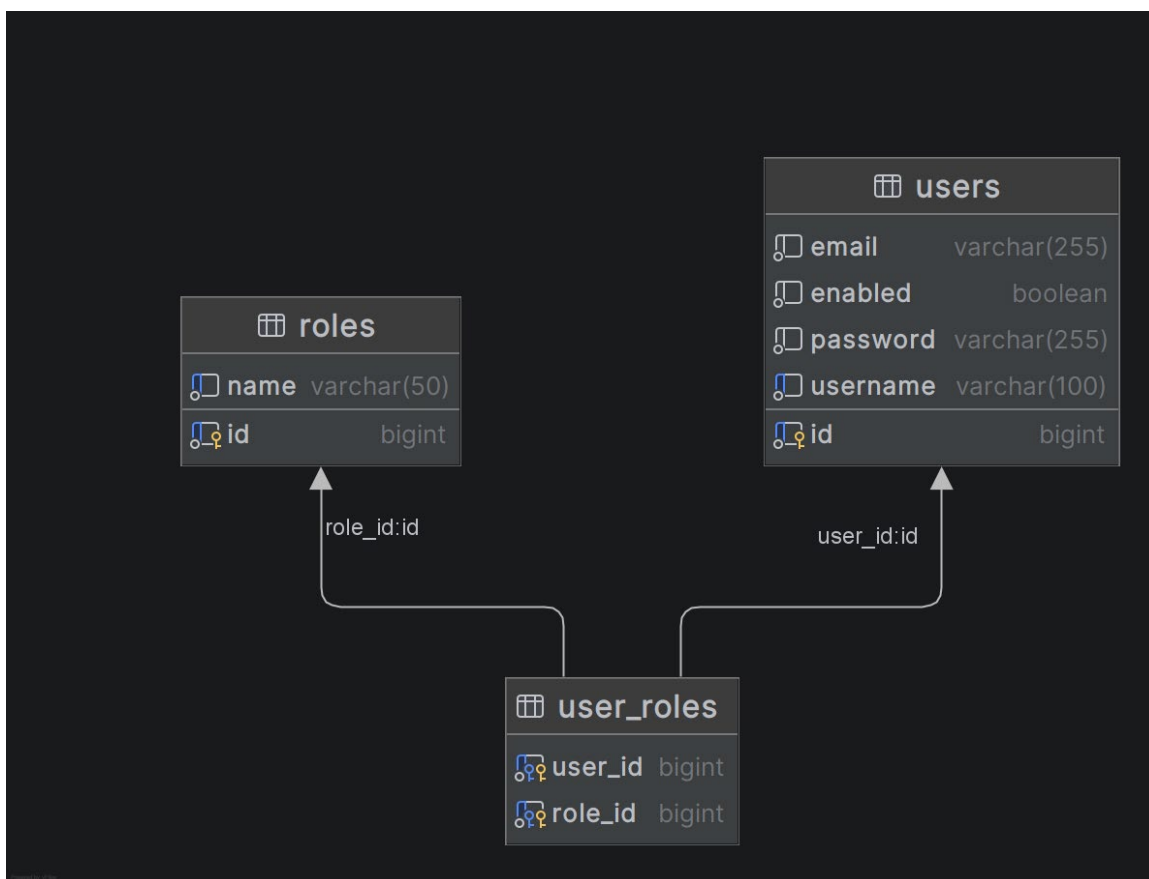


Рисунок 2 – База данных taskboard_auth

Таблица roles содержит поля:

1. id — числовой первичный ключ, назначаемый при вставке строки.
2. name — перечислимое имя роли (ADMIN, MANAGER, EXECUTOR), уникальное в таблице.

Таблица users содержит поля:

1. id — идентификатор учётной записи, далее используемый во всех сервисах как внешний ключ логики.
2. username — уникальное имя входа, ограниченное по длине.
3. password — строка хеша BCrypt, не предназначенная для обратного восстановления.
4. email — адрес почты, участвующий в профиле и уникальности.

5. enabled — признак активности учётной записи.

Связь многие-ко-многим между пользователями и ролями выражена таблицей user_roles с парой внешних ключей на users и roles.

На рисунке 3 показана структура базы taskboard_tasks, принадлежащей сервису задач.

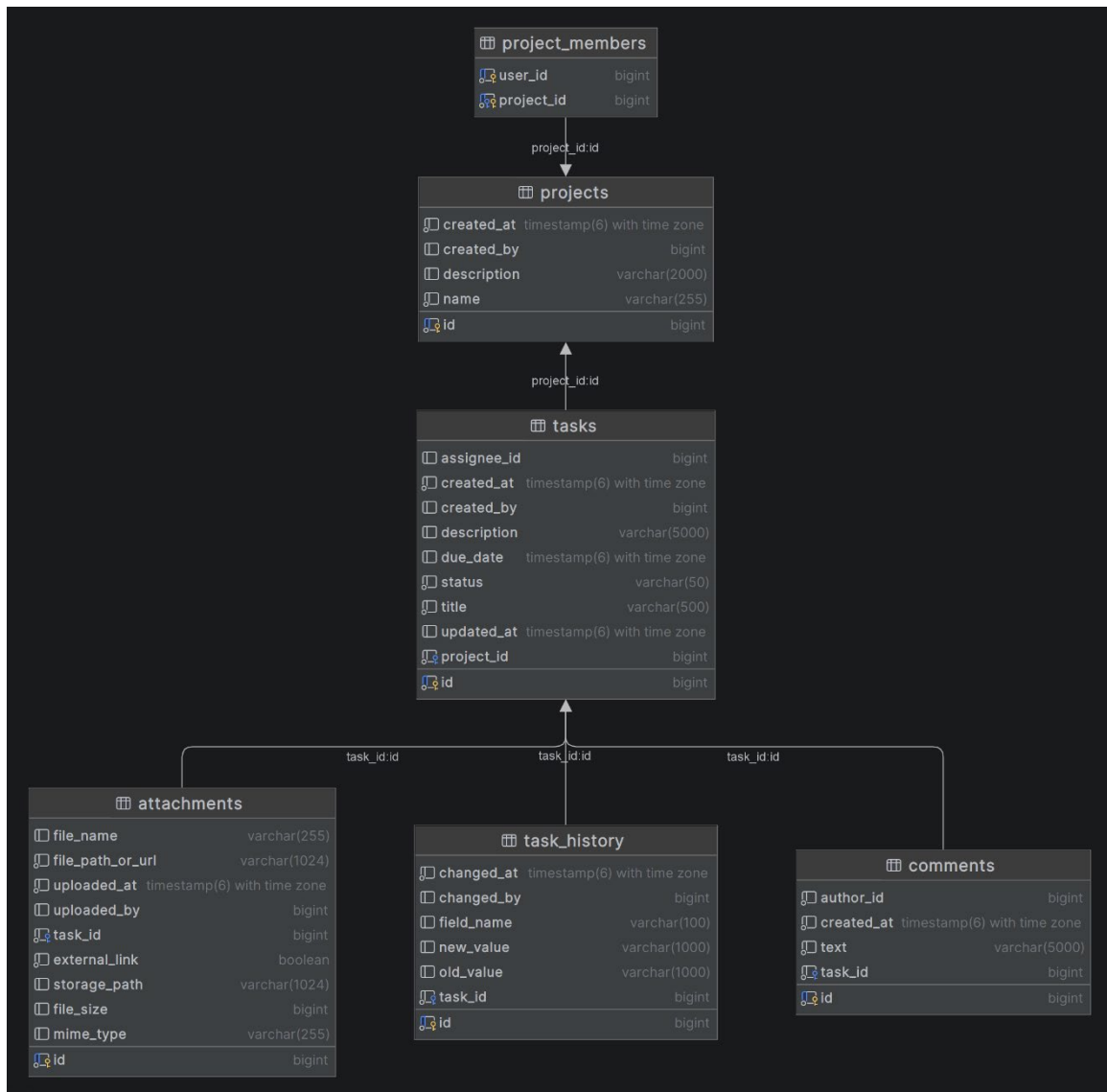


Рисунок 3 – База данных taskboard_tasks

Таблица projects содержит поля:

1. id — числовой первичный ключ, назначаемый при вставке строки.
2. name — наименование проекта, обязательное.

3. `description` — текстовое описание ограниченной длины, может быть пустым.

4. `created_at` — момент создания записи в формате UTC.

5. `created_by` — идентификатор пользователя-автора проекта в терминах сервиса аутентификации.

Таблица `project_members` фиксирует членство в проекте:

1. `project_id` — внешний ключ на `projects`.

2. `user_id` — идентификатор пользователя из базы `taskboard_auth`.

Составной первичный ключ по паре полей `project_id` и `user_id` не допускает повторного вхождения одного и того же пользователя в один проект.

Таблица `tasks` содержит поля:

1. `id` — первичный ключ задачи.

2. `title` — краткий заголовок.

3. `description` — развёрнутое описание.

4. `status` — строковое перечисление стадий жизненного цикла: `OPEN`, `IN_PROGRESS`, `REVIEW`, `DONE`, `CANCELLED`.

5. `assignee_id` — исполнитель или пустое значение, если назначение отсутствует.

6. `due_date` — контрольный срок, необязательный.

7. `created_at`, `updated_at` — временные метки создания и последнего изменения.

8. `created_by` — автор карточки задачи.

9. `project_id` — обязательная связь с проектом.

Таблица `comments` хранит идентификатор, текст, автора (поле `user_id`), задачу и момент создания. Таблица `attachments` хранит имя файла для отображения, MIME-тип, размер в байтах, ключ объекта в MinIO, кто загрузил, и ссылку на задачу. Таблица `task_history` содержит имя изменённого

атрибута, предыдущее и новое значение в текстовом виде, время фиксации и инициатора; записи добавляются при штатных операциях изменения задачи и не предназначены для правки с экрана.

На рисунке 4 приведена структура базы taskboard_analytics, используемой сервисом аналитики и уведомлений.

notifications	
body	varchar(4000)
created_at	timestamp(6) with time zone
read	boolean
title	varchar(255)
type	varchar(100)
user_id	bigint
id	bigint

Рисунок 4 – База данных taskboard_analytics

Таблица notifications содержит получателя (поле user_id), строковый тип события, заголовок, развёрнутый текст, признак прочтения и время создания. Отчётные агрегаты в учебном прототипе не материализованы отдельными таблицами: показатели вычисляются запросами к сервису задач и к локальной таблице уведомлений на лету, что снижает риск рассинхронизации копий на этапе MVP.

Распределённая согласованность между базами не оформляется двухфазными транзакциями: если запись задачи прошла, а вызов уведомления временно не удался, в промышленной системе потребовались бы повторные попытки или очередь сообщений; в рамках работы зафиксировано устойчивое выполнение штатных сценариев при исправной сети между контейнерами. Идентификаторы пользователей и проектов согласованы соглашением: везде используются те же целочисленные значения, что выдали auth-service и tasks-service при создании сущностей.

2.4 Сервис аутентификации и управления учётными записями

Сервис auth-service реализован на Spring Boot 3 (версия платформы в родительском POM — 3.2.0) под управлением Java 17. Доступ к данным организован через Spring Data JPA; в учебном контуре схема базы может выравниваться средствами ORM с последующей фиксацией отдельными скриптами при подготовке к эксплуатации. Контроллеры не тащат в себя правила предметной области: они принимают HTTP-запрос, передают работу сервису регистрации или профиля и отдают DTO, не раскрывая детали таблиц.

При регистрации проверяется уникальность имени и почты; пароль до записи в таблицу users проходит через BCrypt. Первый успешно зарегистрированный пользователь получает роль администратора — это упрощает развёртывание пустого стенда без ручной вставки строк в БД. Последующие регистрации по умолчанию получают роль исполнителя; изменение ролей выполняет администратор через отдельную конечную точку API.

Вход по паре логин и пароль сопоставляется с записью в users, проверяется хеш, из набора ролей формируется список имён для включения в JWT. Токен подписывается симметричным ключом HS256; срок жизни и секрет задаются конфигурацией Spring. Отдельный маршрут позволяет проверить валидность токена — это удобно для отладки и для согласования с

внешними проверками, хотя основной потребитель проверки — приложения tasks-service и analytics-service в роли серверов ресурсов.

Профиль текущего пользователя читается и обновляется по маршрутам с префиксом /api/auth/me; смена пароля вынесена в отдельную операцию с проверкой текущего пароля и валидацией длины нового. Список пользователей для интерфейса назначения исполнителей и для административной таблицы ролей отдаётся отдельным ресурсом: в ответе присутствуют идентификатор, имя, почта и перечень ролей без раскрытия хеша.

Административное изменение ролей адресовано URI с идентификатором пользователя; доступ к операции ограничен на уровне Spring Security. Auth-service ограничен учётом личности и ролями и не знает о карточках задач: при необходимости тот же модуль можно было бы подключить в другом корпоративном контуре, сохранив формат JWT.

Функционал auth-service в терминах предоставляемых операций:

1. Регистрация учётной записи с валидацией полей и назначением стартовой роли по правилу «первый пользователь — администратор».
2. Аутентификация по логину и паролю с выдачей JWT, содержащего идентификатор и роли.
3. Проверка токена на отдельном маршруте для диагностики и вспомогательных сценариев.
4. Чтение и обновление профиля текущего пользователя; смена пароля с проверкой старого значения.
5. Выдача списка пользователей для клиентских экранов без раскрытия хеша пароля.
6. Изменение набора ролей выбранного пользователя администратором.

Этим исчерпывается зона auth-service: вопрос «допускается ли правка задачи в проекте X» решается в tasks-service по полям и membership из того же токена.

2.5 Сервис проектов, задач, комментариев, вложений и истории

Tasks-service — это рабочий контур прототипа: проекты, поручения, обсуждения, файлы и журнал правок. Здесь те же Java 17, Spring Boot 3, JPA и Spring Security с JWT из taskboard-common, что и в остальных backend-узлах. Входящий запрос в контроллере сводится к разбору параметров и вызову либо сервиса чтения, либо командного сервиса; правила доступа и ветвления сценария сосредоточены ниже HTTP-слоя, их проще сопровождать и автоматически проверять.

Доступ к данным проектов и задач согласован с ролевой моделью и таблицей участия project_members. Исполнитель с ролью EXECUTOR при запросе списков видит только те проекты, где он записан участником; попытка отфильтровать выдачу по чужому проекту приводит к пустой странице результата, а не к утечке чужих карточек. Администратор и менеджер получают более широкий обзор по правилам, закреплённым в коде проверок. Отдельно проверяется согласованность назначения: если для задачи указан исполнитель, он обязан состоять в том же проекте; иначе API возвращает ошибку с пояснением, которое интерфейс может показать пользователю без дополнительной интерпретации.

Операции над проектами включают создание, изменение, удаление и выборку с учётом прав текущего субъекта. Участники проекта ведутся во вспомогательной сущности: добавление пользователя в проект и исключение из него оформлены отдельными запросами; состав участников не смешивается с полем исполнителя в задаче, хотя логика назначения ими опирается.

Задачи поддерживают полный цикл в терминах учебного прототипа: создание, правка атрибутов, смена статуса, снятие или назначение

исполнителя, привязка к проекту, контрольный срок. Списки задач допускают фильтрацию по проекту, статусу и исполнителю, а также постраничный вывод, чтобы не перегружать ответ при росте числа записей. Удаление задачи предусмотрено для штатного ведения бэклога.

Комментарии оформлены как дочерние сущности задачи: просмотр ленты, добавление и удаление собственных записей с проверкой авторства и членства в проекте. Это удерживает обсуждение в контуре той же политики доступа, что и сама карточка.

Вложения хранятся двухкомпонентно: метаданные и ссылка на объект в MinIO лежат в PostgreSQL базы `taskboard_tasks`, а байты файла — в объектном хранилище. Загрузка идёт через отдельную конечную точку с типом `multipart`; выдача файла пользователю выполняется через защищённые маршруты скачивания и предпросмотра, чтобы обойти прямой доступ к MinIO из браузера. Удаление вложения согласует запись в базе и объект в хранилище в рамках сценария сервиса. Предпросмотр используется для типов, которые допустимо показать `inline`, в том числе для изображений и текстовых форматов в объёме реализации.

История изменений задачи накапливается в отдельной таблице: фиксируются имя атрибута, предыдущее и новое значение в текстовом виде, время события и инициатор. Записи порождаются при штатных операциях изменения задачи; ручного редактирования истории с экрана не предусмотрено, что сохраняет трассу для разбора спорных ситуаций и отчётности.

После значимых действий в домене задач сервис инициирует внутреннее обращение к `analytics-service` для фиксации уведомления у заинтересованных пользователей. Канал защищён общим секретом и не предназначен для вызова из браузера, что отделяет публичный API от межсервисного.

Сводный перечень внешне доступных групп операций tasks-service в терминах предметной области:

1. Ведение проектов: просмотр доступных проектов, создание, обновление, удаление согласно ролям.
2. Управление участниками проекта: просмотр состава, приглашение пользователя, исключение пользователя.
3. Ведение задач: создание, изменение, удаление, фильтрация и постраничный список.
4. Комментарии к задаче: чтение ленты, добавление, удаление с проверкой прав.
5. Вложения: список, загрузка, скачивание, предпросмотр, удаление при сохранении целостности с MinIO.
6. История: выборка последних записей изменений с ограничением объёма выборки.
7. Инициирование служебных уведомлений во внутреннем контуре analytics-service после событий в задачах.

2.6 Сервис уведомлений, отчётности и экспорта данных

Analytics-service идёт следом за фиксацией факта в tasks-service: принимает служебные сообщения, складывает персональные уведомления, отдаёт их в браузер, по запросам собирает сводки по данным задач и готовит файл выгрузки. База taskboard_analytics изолирована от таблиц задач; актуальные агрегаты по задачам получают запросами к публичным read-only интерфейсам tasks-service, чтобы не дублировать первичные факты и не вводить вторую запись одной и той же сущности.

Уведомления адресуются конкретному пользователю по числовому идентификатору из JWT. Любая выборка и отметка прочтения выполняется в границах текущего субъекта: просмотреть чужую ленту нельзя. При создании уведомления из внешнего API идентификатор получателя не подставляется

из произвольного поля тела запроса недоверенным клиентом — он выводится из контекста аутентификации, что снижает риск подмены адресата.

Для клиента доступны фильтры по признаку прочтения, типу события и текстовому поиску; отдельно поддерживается подсчёт непрочитанных для индикатора в шапке. Отметка прочтения выполняется точечным обновлением записи. Отдельно поднят поток Server-Sent Events: после рукопожатия сервер по мере записи новых строк отдаёт события клиенту, счётчик и лента обновляются без циклического запроса полного списка.

Межсервисное создание уведомления выполняется отдельной конечной точкой и требует заголовок с разделяемым ключом, согласованным в конфигурации обоих сервисов. Тело несёт тип события, заголовок и текст; сопоставление с пользователями выполняет доверенный код на стороне инициатора события в tasks-service.

Отчётная часть опирается на временной интервал с параметрами начала и конца периода. Сводный отчёт даёт обобщённые показатели по задачам за выбранный срез. Разрез по проектам и по исполнителям позволяет сравнить вклад и загрузку без выгрузки сырой таблицы вручную. Экспорт в табличный файл ориентирован на открытие в Excel: кодировка UTF-8 с меткой порядка байтов, русскоязычные заголовки колонок и пригодный для анализа формат даты и времени.

Перечень ключевых функций analytics-service:

1. Хранение и выдача уведомлений с фильтрацией и поиском.
2. Подсчёт непрочитанных и отметка записи прочитанной.
3. Поточковая доставка новых уведомлений по SSE для авторизованного пользователя.
4. Приём внутренних уведомлений от tasks-service по защищённому каналу.

5. Формирование сводного отчёта, отчёта по проектам и отчёта по исполнителям за период.

6. Выгрузка отчёта в CSV для последующей обработки в табличном редакторе.

2.7 Веб-клиент: маршруты, состояние, взаимодействие с API

Клиентская часть прототипа — одностраничное приложение на React 18 со сборкой Vite и маршрутизацией React Router. Vite ускоряет пересборку при правках, Router разносит экраны по модулям без лишней связности. Обращения к трём backend-сервисам выполняются с браузера по HTTP; адреса API в режиме разработки задаются переменными среды сборщика и согласуются с портами контейнеров, а проверка подлинности запросов опирается на JWT в заголовке Authorization.

После входа пользователь попадает на панель обзора с краткими показателями и переходами в основные разделы. Раздел проектов поддерживает создание и редактирование карточек, просмотр списка и переход к деталям выбранного проекта. На странице проекта отображаются задачи этого проекта, фильтры, форма новой задачи и блок управления участниками для ролей, которым это разрешено; состав участников подгружается и обновляется без перезагрузки всего приложения.

Самостоятельный раздел задач даёт единый список с фильтрами по проекту, статусу и исполнителю; отсюда же создаются новые поручения. Карточка задачи объединяет редактирование полей, ленту комментариев, сетку вложений с предпросмотром и журнал изменений; при выборе исполнителя список кандидатов ограничен участниками проекта, что совпадает с проверкой на сервере.

Раздел аналитики оформлен в той же визуальной системе, что и профиль: выделенный верхний блок с ключевыми числами, выбор периода и быстрые пресеты длительности, переключаемые представления распределения статусов, блок динамики относительно предыдущего

интервала той же длины, рейтинги проектов и исполнителей, кнопка выгрузки CSV. Раздел уведомлений реализует ленту с фильтрами по прочитанности и типу, текстовым поиском по содержимому, развёртыванием записи для чтения полного текста и массовой отметкой прочитанными; при раскрытии записи она может быть помечена прочитанной для согласования с серверной моделью.

Профиль отображает данные учётной записи и форму смены пароля. Для администратора добавлен сворачиваемый блок управления ролями: список пользователей подгружается при первом раскрытии, поддерживаются поиск по имени и почте и постраничный просмотр при большом числе записей. Экраны входа и регистрации завершают набор маршрутов открытой части.

На уровне взаимодействия с пользователем применены проверки полей форм до отправки, всплывающие сообщения об успехе и ошибке, состояния-заглушки на время запросов и адаптивная верхняя навигация с мобильным меню. Счётчик непрочитанных уведомлений в шапке сочетается с периодическим опросом и подпиской на SSE, чтобы данные не устаревали при длительной работе с интерфейсом.

Локальная проверка логики вынесена в unit-тесты на движке Vitest совместно с Testing Library там, где это даёт устойчивый эффект без избыточной имитации DOM. Скриншоты основных экранов входят в иллюстративные материалы к работе и приводятся в разделе, посвящённом демонстрации интерфейса.

Функциональные возможности клиента в сжатом виде:

1. Регистрация, аутентификация и сохранение сессии с использованием JWT.
2. Обзорная панель и навигация по разделам приложения.
3. Работа с проектами и участниками, переход к задачам в контексте проекта.

4. Общий реестр задач с фильтрацией и созданием поручений.
5. Детальная карточка задачи с комментариями, файлами и историей.
6. Аналитика по периоду с графическими представлениями и выгрузкой CSV.
7. Лента уведомлений с фильтрами и операциями прочтения.
8. Профиль, смена пароля и административное назначение ролей в пределах полномочий.

2.8 Контейнеризация, конфигурирование окружения и журналирование

Развёртывание прототипа автоматизировано средствами Docker и Docker Compose: каждый backend-сервис получает собственный контейнер приложения и отдельный контейнер PostgreSQL; объектное хранилище MinIO поднимается отдельным сервисом. На ноутбуке разработчика поднимается тот же набор контейнеров, что и при показе работы; отдельно ставить PostgreSQL на хост не требуется. Сборка образов для tasks-service и analytics-service выполняется с контекстом корня репозитория, чтобы в образ попадал общий модуль taskboard-common.

Порты публикации на хост вынесены в переменные окружения: при отсутствии переопределений auth-service доступен на 8081, tasks-service на 8082, analytics-service на 8083, экземпляры PostgreSQL занимают соседние порты 5432–5434, MinIO — 9000 и консоль управления на 9001. Для устранения коллизий с ранее заданными в сеансе значениями в репозитории приведён файл с фиксированными значениями по умолчанию, который можно передать композеру явно. Секреты JWT, пароли баз, ключи доступа к MinIO и внутренний ключ уведомлений задаются через окружение; в корне проекта поставляется пример заполнения, рабочий файл с секретами не включается в систему контроля версий.

Клиент Vite в режиме разработки использует собственный файл с базовыми URL API на локальные порты сервисов, чтобы разработчик мог не копировать настройки вручную при стандартном сценарии. При необходимости нестандартных адресов предусмотрено локальное переопределение без изменения общих шаблонов.

Журналирование backend-узлов настроено на запись в файлы с ротацией в каталоге logs с разделением по имени сервиса. В запись включаются идентификатор запроса для сквозной трассировки, идентификатор пользователя при наличии аутентификации, метод и путь, код ответа и длительность обработки. Уровень детализации задаётся параметром окружения. Для ускорения разбора инцидентов в корне репозитория имеется вспомогательный сценарий на PowerShell, позволяющий просматривать хвост файлов, следить за потоком в реальном времени и отбирать строки по идентификатору запроса или пользователя.

Конфигурация CORS на tasks-service и analytics-service согласована с отдельным происхождением фронтенда и API в учебной среде: браузеру разрешено обращаться к серверам с origin dev-сервера Vite при условии предъявления действительного JWT на защищённых маршрутах.

2.9 Тестирование и конвейер непрерывной интеграции

Качество проверяют и модульные, и интеграционные тесты backend на Maven и JUnit. Auth-service собирают и гоняют отдельной командой; tasks-service, analytics-service и jar taskboard-common — одним проходом реактора, как в CI, чтобы общая библиотека оказалась в classpath потребителей до старта тестов.

На стороне клиента unit-тесты запускаются командой `npm run test` во фронтенд-каталоге и покрывают устойчивые к автоматизации фрагменты: утилиты разбора данных уведомлений, хуки пагинации и сопутствующую логику без тяжёлого e2e-контура. End-to-end сценарии на Playwright [9] проверяют сквозной пользовательский поток в браузере против поднятого

стека Docker Compose и dev-сервера Vite; в прогоне задаются базовый URL интерфейса и базовые адреса API, чтобы сценарии не зависели от жёстко прошитых портов хоста вне согласованного окружения.

Локальный сводный сценарий на PowerShell объединяет прогон backend, при полном режиме — установку зависимостей фронтенда и unit-тесты, а при необходимости — подготовку изолированного compose-стека и e2e. Скрипт контролирует код возврата Maven, наличие признаков сбоя в журнале сборки и итоговые счётчики тестов по отчёту Surefire, чтобы не принять частично упавший прогон за успешный. Результаты и детальные журналы складываются в подкаталог logs с меткой времени запуска, что облегчает разбор регрессий.

Конвейер непрерывной интеграции на GitHub Actions [10] разбит на две последовательные задачи. Сначала на средстве выполнения с Ubuntu поднимается Java 17, кэшируются зависимости Maven, выполняются тесты auth-service и затем совместный прогон tasks-service с analytics-service и общим модулем. После успешного завершения backend-задачи запускается вторая: устанавливается Node.js, поднимается docker compose up с пересборкой образов, во фронтенде выполняются npm install и Vitest, устанавливается Chromium для Playwright, в фоне стартует dev-сервер с прослушиванием на всех интерфейсах, ожидается готовность порта 3000, затем выполняется npm run test:e2e с переменными окружения, указывающими базовый URL приложения и префиксы API трёх сервисов. При сбое прикладывается журнал фронтенд-сервера для анализа.

На виртуальной машине GitHub Actions без предварительной ручной настройки выполняется тот же сценарий, что проверялся у автора; это отсекает часть регрессий по контракту API и по базовому пользовательскому прохождению до слияния в основную ветку.

3 РЕЗУЛЬТАТ РАБОТЫ

3.1 Описание стенда и фрагмент Docker Compose

Рабочий стенд MVP поднимается из корня репозитория командой `docker compose up` с пересборкой образов. В полном файле объявлены три экземпляра PostgreSQL (по одному на каждый backend), сервис `auth-service`, объектное хранилище MinIO, а также `tasks-service` и `analytics-service` с общим секретом JWT и отдельным ключом для внутренних уведомлений. Порты и пароли задаются через переменные окружения с разумными значениями по умолчанию; при коллизии портов на хосте используется поставляемый файл `docker.defaults.env`. Клиент React в режиме разработки запускается отдельно и обращается к API по локальным адресам, согласованным с Compose.

На листинге 1 показаны фрагменты `minio`, `tasks-service` и `analytics-service` и объявление томов; остальные сервисы объявлены в том же файле аналогично и связаны через `depends_on` и имя сети по умолчанию Compose.

Листинг 1 – Фрагмент `docker-compose.yml` (MinIO, прикладные сервисы и тома)

```
minio:
  image: minio/minio:latest
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${MINIO_ACCESS_KEY:-minioadmin}
    MINIO_ROOT_PASSWORD: ${MINIO_SECRET_KEY:-minioadmin}
  ports:
    - "${MINIO_PORT:-9000}:9000"
    - "${MINIO_CONSOLE_PORT:-9001}:9001"
  volumes:
    - minio_data:/data

tasks-service:
  build:
    context: .
    dockerfile: tasks-service/Dockerfile
  ports:
    - "${TASKS_SERVICE_PORT:-8082}:8082"
  volumes:
    - ./logs/tasks-service:/app/logs
  environment:
    POSTGRES_HOST: postgres-tasks
```

```

    POSTGRES_PORT: 5432
    POSTGRES_DB: ${POSTGRES_TASKS_DB:-taskboard_tasks}
    POSTGRES_USER: ${POSTGRES_TASKS_USER:-postgres}
    POSTGRES_PASSWORD: ${POSTGRES_TASKS_PASSWORD:-postgres}
    JWT_SECRET: ${JWT_SECRET:-taskboard-auth-secret-key-min-
256-bits-for-hs256}
    ANALYTICS_BASE_URL: http://analytics-service:8083
    NOTIFICATIONS_INTERNAL_KEY:
${NOTIFICATIONS_INTERNAL_KEY:-taskboard-internal-key}
    MINIO_ENDPOINT: http://minio:9000
    MINIO_ACCESS_KEY: ${MINIO_ACCESS_KEY:-minioadmin}
    MINIO_SECRET_KEY: ${MINIO_SECRET_KEY:-minioadmin}
    MINIO_BUCKET: ${MINIO_BUCKET:-taskboard-attachments}
    LOG_FILE: /app/logs/tasks-service.log
    LOG_LEVEL: ${LOG_LEVEL:-INFO}
depends_on:
    postgres-tasks:
        condition: service_healthy
    minio:
        condition: service_started

analytics-service:
    build:
        context: .
        dockerfile: analytics-service/Dockerfile
    ports:
        - "${ANALYTICS_SERVICE_PORT:-8083}:8083"
    volumes:
        - ./logs/analytics-service:/app/logs
    environment:
        POSTGRES_HOST: postgres-analytics
        POSTGRES_PORT: 5432
        POSTGRES_DB: ${POSTGRES_ANALYTICS_DB:-
taskboard_analytics}
        POSTGRES_USER: ${POSTGRES_ANALYTICS_USER:-postgres}
        POSTGRES_PASSWORD: ${POSTGRES_ANALYTICS_PASSWORD:-
postgres}
        JWT_SECRET: ${JWT_SECRET:-taskboard-auth-secret-key-min-
256-bits-for-hs256}
        NOTIFICATIONS_INTERNAL_KEY:
${NOTIFICATIONS_INTERNAL_KEY:-taskboard-internal-key}
        TASKS_BASE_URL: http://tasks-service:8082
        AUTH_BASE_URL: http://auth-service:8081
        LOG_FILE: /app/logs/analytics-service.log
        LOG_LEVEL: ${LOG_LEVEL:-INFO}
    depends_on:
        postgres-analytics:
            condition: service_healthy

volumes:
    pg_auth_data:
    pg_tasks_data:

```

```
pg_analytics_data:
minio_data:
```

Здесь видно, что tasks-service знает внутренний HTTP-адрес analytics-service для вызова API уведомлений, а также параметры подключения к MinIO и PostgreSQL; analytics-service, в свою очередь, может обращаться к задачам и к сервису учётных записей для отчётности. Журналы монтируются в каталог logs на хосте, что упрощает разбор инцидентов без входа в контейнер.

3.2 Контроллер проектов и делегирование прикладным сервисам

В сервисе задач HTTP-слой сведён к маршрутам Spring MVC [11]: проверки членства в проекте, валидация DTO и транзакции сосредоточены в ProjectQueryService и ProjectCommandService. Контроллер остаётся «тонким» и удобен для сопоставления с таблицей маршрутов в документации API.

Листинг 2 – Класс ProjectController (фрагмент)

```
@RestController
@RequestMapping("/api/projects")
@RequiredArgsConstructor
public class ProjectController {

    private final ProjectQueryService projectQueryService;
    private final ProjectCommandService projectCommandService;

    @GetMapping
    public List<Project> list(Authentication auth) {
        return projectQueryService.list(auth);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Project> get(@PathVariable Long id,
Authentication auth) {
        return projectQueryService.get(id, auth);
    }

    @PostMapping
    public ResponseEntity<Project> create(@Valid @RequestBody
ProjectWriteDto dto, Authentication auth) {
        return projectCommandService.create(dto, auth);
    }
}
```

Остальные методы контроллера (изменение, удаление, состав участников) построены по той же схеме: аннотация маршрута, извлечение идентификаторов из URI и делегирование сервису команды или запроса.

3.3 Ограничение выборки задач для роли исполнителя

Роль EXECUTOR в учебном прототипе сопоставлена с правилом видимости: исполнитель получает задачи только тех проектов, где он записан в `project_members`. При запросе списка с фильтром по проекту, к которому пользователь не относится, возвращается пустая страница результатов, чтобы не подсказывать наличие чужих данных.

Листинг 3 – Метод `listTasks` в `TaskQueryService` (фрагмент)

```
public ResponseEntity<?> listTasks(Long projectId,
                                   Task.TaskStatus status,
                                   Long assigneeId,
                                   int page,
                                   int size,
                                   Authentication auth) {
    Long userId = RoleAuthorization.userId(auth);
    boolean isExecutor = RoleAuthorization.isExecutor(auth);
    var pageable = PageRequest.of(page, size);

    if (isExecutor) {
        List<Long> allowedProjectIds =
projectMemberRepository.findProjectIdsByUserId(userId);
        if (allowedProjectIds == null ||
allowedProjectIds.isEmpty()) {
            return ResponseEntity.ok(Page.empty(pageable));
        }

        if (projectId != null) {
            if (!allowedProjectIds.contains(projectId)) {
                return
ResponseEntity.ok(Page.empty(pageable));
            }
            Page<Task> result =
taskRepository.findByProjectAndFilters(projectId, status,
assigneeId, pageable);
            return ResponseEntity.ok(result);
        }

        Page<Task> result =
taskRepository.findByProjectIdsAndFilters(allowedProjectIds,
status, assigneeId, pageable);
        return ResponseEntity.ok(result);
    }
}
```

```

    }

    if (projectId != null) {
        Page<Task> result =
taskRepository.findByProjectAndFilters(projectId, status,
assigneeId, pageable);
        return ResponseEntity.ok(result);
    }
    if (assigneeId != null) {
        return
ResponseEntity.ok(taskRepository.findByAssigneeId(assigneeId));
    }
    return
ResponseEntity.ok(taskRepository.findAll(pageable));
}

```

Для менеджера и администратора ветка `isExecutor` не срабатывает, и выборка следует общим фильтрам либо полному постраничному списку.

3.4 Внутренний вызов создания уведомления в сервисе аналитики

Публичное API уведомлений ожидает JWT пользователя; межсервисный приём событий из `tasks-service` вынесен на отдельный путь с проверкой заголовка `X-Internal-Key`. Значение ключа совпадает с переменной `NOTIFICATIONS_INTERNAL_KEY` в обоих сервисах и не передаётся в браузер.

Листинг 4 – Приём внутреннего уведомления в `NotificationController`

```

@PostMapping("/internal")
public ResponseEntity<NotificationDto> createInternalWithKey(
    @RequestHeader(value = "X-Internal-Key", required =
false) String headerKey,
    @RequestBody NotificationDto dto) {
    if (headerKey == null || !headerKey.equals(internalKey))
    {
        return ResponseEntity.status(401).build();
    }
    return
notificationApplicationService.createInternal(dto);
}

```

Так разделяются доверенный канал между сервисами и пользовательские запросы; при неверном или отсутствующем ключе клиент получает ответ 401 без тела.

3.5 Файл конвейера непрерывной интеграции

В репозитории настроен workflow GitHub Actions: сначала изолированно гоняются тесты auth-service, затем совместная сборка tasks-service, analytics-service и модуля taskboard-common; после успеха поднимается Compose, ставятся зависимости фронтенда, выполняются Vitest и e2e на Playwright против поднятого стека и dev-сервера Vite.

Листинг 5 – Фрагмент .github/workflows/ci.yml

```
name: CI

on:
  push:
  pull_request:

env:
  FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: "true"

jobs:
  backend-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - name: Set up Java 17
        uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: "17"
          cache: maven

      - name: Auth service tests
        working-directory: auth-service
        run: mvn -B test

      - name: Tasks + analytics (includes taskboard-common)
        run: mvn -B -pl tasks-service,analytics-service -am
test

  e2e:
    runs-on: ubuntu-latest
    needs: backend-tests
    steps:
      - uses: actions/checkout@v5

      - name: Set up Node
        uses: actions/setup-node@v5
        with:
```

```

        node-version: "20"
        cache: npm
        cache-dependency-path: frontend/package-lock.json

- name: Start services (Docker Compose)
  run: docker compose up -d --build

- name: Install frontend deps
  working-directory: frontend
  run: npm install

- name: Frontend unit tests (Vitest)
  working-directory: frontend
  run: npm run test

- name: Install Playwright browsers
  working-directory: frontend
  run: npx playwright install --with-deps chromium

- name: Start frontend
  working-directory: frontend
  run: nohup npm run dev -- --host 0.0.0.0 --port 3000
>/tmp/frontend.log 2>&1 &

- name: Wait for frontend
  run: |
    for i in {1..60}; do
      curl -fsS http://localhost:3000 >/dev/null &&
      sleep 2
    done
    echo "Frontend did not start"; cat
/tmp/frontend.log; exit 1

- name: Run e2e tests
  working-directory: frontend
  env:
    E2E_BASE_URL: http://localhost:3000
    E2E_AUTH_API: http://localhost:8081/api/auth
    E2E_TASKS_API: http://localhost:8082/api
    E2E_ANALYTICS_API: http://localhost:8083/api
  run: npm run test:e2e

```

Ключевой смысл — воспроизводимость: новый участник или проверяющий получают тот же порядок шагов, что и виртуальная машина CI, если выполняют команды из сценария локально.

3.6 Выводы

По совокупности листингов видно, что прототип закрепляет архитектурные решения второй главы на уровне инфраструктуры и кода: границы сервисов отражены в отдельных контейнерах и базах, разграничение ролей — в сервисном слое выборки задач, межсервисное взаимодействие — во внутреннем маршруте с ключом, качество — в разнесённых задачах Maven и e2e-контуре. Сопоставление с техническим заданием подтверждает закрытие основного функционального контура MVP; ограничения остаются теми же, что обсуждались во введении и в разделе про безопасность: один хост Docker без кластера, отсутствие распределённых транзакций между БД, без двухфакторной аутентификации и без нагрузочного тестирования. Направления развития — оркестрация уровня Kubernetes [12] при необходимости отказоустойчивости, брокер или очередь для согласованности уведомлений, расширение аутентификации и предметных функций задач и отчётности — логично продолжают заложенную декомпозицию без переписывания системы в монолит.

ЗАКЛЮЧЕНИЕ

В ходе выпускной квалификационной работы разработан и описан прототип микросервисного веб-приложения для учёта проектов и задач внутри организации. Реализованы три backend-сервиса на Spring Boot с отдельными базами PostgreSQL, хранение вложений в MinIO, веб-клиент на React и Vite, сценарий развёртывания через Docker Compose, журналирование и конвейер непрерывной интеграции с модульными, интеграционными и сквозными проверками. Достигнута цель работы — работоспособный MVP с ролевым доступом, цепочкой поручений, комментариями и файлами, уведомлениями и аналитикой с выгрузкой отчёта. Практическая ценность решения в том, что в одном прототипе собран рабочий набор функций для совместной работы с проектами и задачами: роли пользователей, комментарии, вложения с хранением файлов в MinIO, уведомления, аналитика и выгрузка отчёта в CSV. Систему можно развернуть на своей инфраструктуре, не привязываясь к внешнему облачному сервису, а разнесение логики по микросервисам с отдельными базами данных облегчает дальнейшее развитие: правки в одном узле не затягивают за собой весь комплекс. Запуск стенда по Docker Compose, журналы запросов и автоматические тесты с прогоном в CI помогают быстрее находить ошибки при доработках и удобнее переводить учебный прототип в опытную эксплуатацию.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Фаулер М. Микросервисы. Паттерны разработки и рефакторинга. — СПб.: Питер, 2020.
2. Fielding R. Architectural Styles and the Design of Network-based Software Architectures: диссертация. — UC Irvine, 2000 — URL: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm> (дата обращения 28.03.2026).
3. Набор официальной документации Spring Boot 3 и Spring Security — URL: <https://docs.spring.io/spring-boot/reference/index.html>; <https://docs.spring.io/spring-security/reference/index.html> (дата обращения 08.03.2026).
4. Документация MinIO — URL: <https://min.io/docs/minio/linux/index.html> (дата обращения 27.03.2026).
5. Документация React 18 — URL: <https://react.dev> (дата обращения 15.03.2026).
6. Обзор сетей и оркестрации контейнеров Docker Compose — URL: <https://docs.docker.com/compose/> (дата обращения 02.04.2026).
7. Спецификация JSON Web Token (RFC 7519) — URL: <https://www.rfc-editor.org/rfc/rfc7519> (дата обращения 19.03.2026).
8. Документация PostgreSQL 16 — URL: <https://www.postgresql.org/docs/16/> (дата обращения 25.03.2026).
Документация Playwright — URL: <https://playwright.dev> (дата обращения 05.04.2026).
9. Документация Vitest — URL: <https://vitest.dev> (дата обращения 06.04.2026).
10. Документация GitHub Actions — URL: <https://docs.github.com/actions> (дата обращения 04.04.2026).
11. Документация Spring Framework: веб-подсистема Servlet и Spring MVC — URL: <https://docs.spring.io/spring->

framework/reference/web/webmvc.html (дата обращения 09.04.2026).

12. Документация Kubernetes — URL: <https://kubernetes.io/docs/> (дата обращения 12.04.2026).

ПРИЛОЖЕНИЕ

Адрес репозитория:

<https://github.com/EvgenyMAI/TaskBoard>



Рисунок А.1 — QR-код для перехода к репозиторию с кодом